

تقسیم کار پروژه Batch Processing

Platform با تیم ۶ Frontend، ۲ Backend، و ۱ Team Lead

بله، با ترکیب تیم:

Backend Developer 6

Frontend Developer 2

Team Lead 1

تقسیم کار حتی بهتر می شود، چون می توان بخش Worker Group Management را به عنوان یک ماژول مستقل و جدی جدا کرد.

در معماری شما، Worker Group یک بخش فرعی نیست؛ یکی از ستون های اصلی platform است، چون هدف شما این است که هر تیم مثل سپرده، تسهیلات، کارت، پرداخت، گزارش گیری و ... worker های مستقل خودش را داشته باشد.

تقسیم پیشنهادی کل تیم

Team Lead

Architecture / Governance / Integration

Backend 1

Control Plane & Execution Manager

Backend 2

Worker Framework & Spring Batch Base

Backend 3

Worker Group, Worker Registry & Runtime Management

Backend 4

Partitioning, Sharding & Distributed Execution

Backend 5

Scheduler, Business Calendar, Dependency & SLA

Backend 6

Error Handling, Audit, Reprocess & Security

Frontend 1

Operations Dashboard, Execution, Worker & Partition UI

Frontend 2

Admin UI, Scheduler UI, Error Center, Reprocess & Audit UI

نقش Team Lead

مسئولیت اصلی

تیم لید نباید وارد پیاده سازی روزمره همه مازول ها شود. نقش اصلی او باید حفظ یکپارچگی معماری باشد.

وظایف

- نهایی سازی معماری کلان
- تعریف Core Platform Contract
- تایید data model اصلی
- تعریف state machine ها
- تعریف integration contract بین ماژولها
- مدیریت ADR ها
- review طراحی های حساس
- هماهنگی بین backend و frontend
- هماهنگی با تیم های دامنه مثل سپرده، تسهیلات، پرداخت
- کنترل scope
- تصمیم گیری درباره RabbitMQ/Kafka, deployment, security, observability

خروجی های اصلی Team Lead

- Target Architecture Document
- Execution Model Design
- Core Platform Contract
- Data Model پایه
- API Contract پایه
- Worker Group Strategy
- ADR ها
- Development Roadmap
- Definition of Done

Backend 1: Control Plane & Execution Manager

این نفر مالک مغز اجرایی پلتفرم است.

مسئولیت ها

- مدیریت Job Execution Request
- start / stop / restart job
- مدیریت execution state
- جلوگیری از duplicate execution
- مدیریت businessDate در سطح execution
- ارتباط با Job Registry
- ارتباط با Worker Group برای dispatch
- مدیریت lifecycle اجرای job

ماژول‌ها

- batch-control-plane
- execution-manager
- execution-state-machine
- job-launch-service
- execution-lock-service

جداول مرتبط

- BATCH_EXECUTION_REQUEST
- BATCH_EXECUTION_LOCK
- BATCH_JOB_EXECUTION_STATE
- BATCH_MANUAL_ACTION_AUDIT

API‌های نمونه

- POST /jobs/{jobName}/start
- POST /executions/{executionId}/stop
- POST /executions/{executionId}/restart
- GET /executions
- GET /executions/{executionId}
- GET /executions/{executionId}/timeline

وابستگی‌ها

این نفر باید با این افراد هماهنگ باشد:

Backend 3 برای انتخاب worker group و dispatch
Backend 4 برای sharded execution
Backend 5 برای scheduled execution
Backend 6 برای audit و permission
Frontend 1 برای execution UI

Definition of Done

- job از طریق API قابل اجرا باشد.
- execution state دقیق ذخیره شود.
- duplicate execution برای job/businessDate کنترل شود.
- stop/restart پایه پشتیبانی شود.
- تمام actionها audit شوند.

Backend 2: Worker Framework & Spring Batch Base

این نفر مسئول زیرساخت توسعه jobهاست.

مسئولیت‌ها

- ساخت base library برای jobها
- integration استاندارد با Spring Batch
- تعریف Job Parameters استاندارد
- ساخت Worker Agent
- ساخت job template
- استاندارد Reader / Processor / Writer
- logging و metrics پایه داخل worker
- graceful shutdown در worker

ماژول‌ها

```
batch-worker-core
spring-batch-starter
worker-agent
job-template-library
batch-common-library
```

خروجی‌های مهم

```
BaseBatchJob
BaseItemReader
BaseItemProcessor
BaseItemWriter
BaseAuditWriter
BaseErrorHandler
BusinessDataContext
ExecutionContextPropagator
```

استاندارد Job Parameter

هر job باید حداقل این‌ها را داشته باشد:

```
businessDate
executionId
runId
requestedBy
domain
executionMode
workerGroup
```

وابستگی‌ها

```
Backend 1 برای executionId و launch contract
Backend 3 برای worker registration و worker group config
Backend 4 برای partition execution
Backend 6 برای audit/error استاندارد
```

Definition of Done

- توسعه‌دهنده بتواند با template استاندارد job جدید بنویسد.
- worker بتواند job را با پارامترهای platform اجرا کند.
- businessDate در تمام job قابل دسترس باشد.
- log و metric استاندارد تولید شود.
- worker بتواند graceful stop انجام دهد.

Backend 3: Worker Group, Worker Registry & Runtime Management

این همان بخشی است که باید حتما لحاظ شود.

چرا این نقش مهم است؟

چون هدف این است که به ازای هر تیم worker مستقل داشته باشید:

deposit workers
loan workers
payment workers
card workers
reporting workers

پس باید یک مازول مستقل داشته باشید که بداند:

چه workerهایی زنده هستند؟
هر worker متعلق به کدام worker group است؟
هر worker چه jobهایی را میتواند اجرا کند؟
هر worker چقدر capacity دارد؟
کدام worker busy است؟
کدام worker down شده؟
کدام worker چه نسخه‌ای دارد؟
کدام worker برای چه domain مجاز است؟

مسئولیت‌ها

- طراحی و پیاده‌سازی Worker Group Model
- Worker Registration
- Worker Heartbeat
- Worker Discovery
- Worker Capability Management
- Worker Capacity Management
- Routing job به worker group مناسب
- مدیریت allowed jobs برای هر worker group
- مدیریت worker version
- مدیریت worker health
- مدیریت concurrency limit
- مدیریت drain/disable کردن worker

ماژول‌ها

worker-group-service
worker-registry-service
worker-discovery-service
worker-capacity-manager
worker-routing-service
worker-health-service

جداول مرتبط

BATCH_WORKER_GROUP
BATCH_WORKER_INSTANCE
BATCH_WORKER_CAPABILITY
BATCH_WORKER_HEARTBEAT
BATCH_WORKER_ASSIGNMENT
BATCH_WORKER_GROUP_JOB
BATCH_WORKER_GROUP_LIMIT
BATCH_WORKER_VERSION

مدل پیشنهادی Worker Group

```
BATCH_WORKER_GROUP
-----
worker_group_id
name
domain
workload_type
criticality
max_concurrent_jobs
max_concurrent_partitions
status
owner_team
description
created_at
updated_at
```

نمونه worker group ها:

```
deposit-db-heavy-critical
deposit-file-heavy-critical
loan-db-heavy-critical
payment-api-heavy-critical
card-db-heavy-critical
reporting-readonly
reconciliation-file-heavy
```

مدل Worker Instance

BATCH_WORKER_INSTANCE

worker_id

worker_group_id

hostname

ip

port

status

current_job_count

current_partition_count

max_job_capacity

max_partition_capacity

app_version

last_heartbeat_at

started_at

Worker Status

STARTING

UP

BUSY

DRAINING

DISABLED

DOWN

STALE

UNKNOWN

Worker Group Status

ACTIVE

DISABLED

MAINTENANCE

DEGRADED

قابلیت‌های مهم

Worker Registration .1

وقتی worker بالا می‌آید، خودش را register می‌کند:

```
workerName  
workerGroup  
domain  
capabilities  
version  
host  
maxConcurrency  
supportedJobNames
```

Worker Heartbeat .2

هر worker در بازه مشخص heartbeat می‌فرستد:

```
workerId  
status  
activeJobs  
activePartitions  
cpuUsage  
memoryUsage  
dbPoolUsage  
lastError
```

Worker Routing .3

وقتی job اجرا می‌شود، سیستم باید تشخیص دهد کدام worker group مناسب است.

مثلا:

```
jobName: deposit-interest-calculation  
domain: DEPOSIT  
workloadType: DB_HEAVY  
executionMode: SHARDED  
requiredWorkerGroup: deposit-db-heavy-critical
```

Worker Drain .4

برای deployment یا maintenance باید بتوان worker را از UI drain کرد:

DRAINING یعنی:

- job جدید به worker داده نشود
- jobهای فعلی تمام شوند
- بعد worker قابل shutdown باشد

Version Awareness .5

خیلی مهم است که بدانید کدام worker با چه نسخه‌ای اجرا می‌شود.

مثلا:

```
deposit-worker-01 -> version 1.4.2
deposit-worker-02 -> version 1.4.2
deposit-worker-03 -> version 1.3.9
```

در UI باید مشخص باشد.

APIهای نمونه

```
POST /workers/register
POST /workers/{workerId}/heartbeat
GET /worker-groups
GET /worker-groups/{groupId}
GET /worker-groups/{groupId}/workers
POST /worker-groups/{groupId}/disable
POST /worker-groups/{groupId}/enable
POST /workers/{workerId}/drain
POST /workers/{workerId}/disable
GET /workers/{workerId}/health
```

وابستگی‌ها

Backend 1 برای dispatch execution به worker group
Backend 2 برای worker agent و heartbeat
Backend 4 برای تخصیص partition به workerها
Backend 5 برای scheduler capacity check
Frontend 1 برای Worker Monitor UI
Frontend 2 برای Worker Group Admin UI

Definition of Done

- workerها بتوانند register شوند.
- heartbeat workerها ثبت شود.
- وضعیت worker در UI قابل مشاهده باشد.
- worker groupها قابل تعریف و مدیریت باشند.
- job فقط به worker group مجاز ارسال شود.
- concurrency limit رعایت شود.
- worker down/stale تشخیص داده شود.
- worker قابل disable/drain باشد.

Backend 4: Partitioning, Sharding & Distributed Execution

این نفر مسئول اجرای حجیم و distributed است.

مسئولیت‌ها

- طراحی Partition Planner
- تقسیم داده برای job های +100M
- پیاده سازی local partitioning
- پیاده سازی remote partitioning
- ایجاد partition plan
- claim کردن partition توسط worker
- retry failed partition
- aggregate نتیجه partition ها
- مدیریت stale partition

ماژول ها

```
partition-planner
partition-execution-service
remote-partition-manager
partition-assignment-service
partition-result-aggregator
```

جداول مرتبط

```
BATCH_PARTITION_PLAN
BATCH_PARTITION_EXECUTION
BATCH_PARTITION_LOCK
BATCH_PARTITION_RESULT
```

ارتباط با Worker Group

این بخش باید با 3 Backend هماهنگ باشد.

چون partition ها باید فقط به worker های همان worker group ارسال شوند.

مثلا:

```
Job: deposit-interest-calculation
Worker Group: deposit-db-heavy-critical
Partitions: 128
Available Workers: 8
Max partitions per worker: 4
```

پس حداکثر همزمان:

```
partitions 32 = 4 * 8
```

اجرا می‌شوند.

Partition State

```
CREATED
ASSIGNED
CLAIMED
RUNNING
COMPLETED
FAILED
RETRY_PENDING
CANCELLED
STALE
```

API های نمونه

```
GET /executions/{executionId}/partitions
GET /partitions/{partitionId}
POST /partitions/{partitionId}/retry
POST /partitions/{partitionId}/cancel
GET /partition-plans/{planId}
```

Definition of Done

- job بتواند به partition های کنترل شده تقسیم شود.
- partition ها بین worker های مجاز توزیع شوند.
- شکست یک partition کل job را مبهم نکند.
- retry فقط روی partition های failed ممکن باشد.
- وضعیت partition در UI قابل مشاهده باشد.

Backend 5: Scheduler, Business Calendar, Dependency & SLA

این نفر مسئول زمان بندی بانکی است.

مسئولیت ها

- scheduler داخلی
- cron schedule
- business calendar
- holiday calendar
- محاسبه businessDate
- dependency بین job ها
- EOD/BOD chain
- misfire handling
- SLA monitoring
- capacity check قبل از trigger job

ماژول ها

batch-scheduler
business-calendar-service
job-dependency-engine
sla-monitor
schedule-trigger-service

جداول مرتبط

BATCH_SCHEDULE
BATCH_BUSINESS_CALENDAR
BATCH_JOB_DEPENDENCY
BATCH_SLA_POLICY
BATCH_SCHEDULE_TRIGGER

ارتباط با Worker Group

Scheduler نباید کورکورانه job را trigger کند.

قبل از اجرای job باید بررسی کند:

- worker group مربوطه active است؟
- حداقل worker سالم وجود دارد؟
- capacity کافی وجود دارد؟
- job مشابه برای همان businessDate در حال اجرا نیست؟
- dependencyها کامل شده اند؟

مثال:

اگر deposit-db-heavy-critical در حالت MAINTENANCE باشد، scheduler نباید jobهای سپرده را trigger کند.

Definition of Done

- job طبق schedule اجرا شود.
- businessDate درست محاسبه شود.
- dependencyها enforce شوند.
- SLA breach تشخیص داده شود.
- scheduler وضعیت worker group را قبل از اجرا لحاظ کند.

Backend 6: Error Handling, Audit, Reprocess & Security

این نفر مسئول کنترل‌های حساس بانکی است.

مسئولیت‌ها

- error model
- error classification
- retry policy
- reprocess flow
- approval flow
- business audit
- manual action audit
- RBAC
- permission enforcement
- masking داده حساس

ماژول‌ها

- error-center-service
- audit-service
- reprocess-service
- approval-service
- security-service
- permission-service

جداول مرتبط

- BATCH_ERROR_EVENT
- BATCH_REPROCESS_REQUEST
- BATCH_REPROCESS_RESULT
- BATCH_APPROVAL_REQUEST
- BATCH_BUSINESS_AUDIT
- BATCH_MANUAL_ACTION_AUDIT
- BATCH_ROLE
- BATCH_PERMISSION
- BATCH_USER_ROLE

ارتباط با Worker Group

بعضی action ها باید در سطح worker group هم audit و permission داشته باشند:

- disable worker group
- enable worker group
- drain worker
- retry partition روی worker group خاص
- start job روی worker group خاص

مثلا فقط owner تیم سپرده بتواند job های worker group سپرده را اجرا یا retry کند.

Definition of Done

- خطاها ذخیره و دسته بندی شوند.
- reprocess با approval انجام شود.
- تمام عملیات حساس audit شود.
- permission ها در API enforce شوند.
- داده حساس mask شود.

Frontend 1: Operations Dashboard, Execution, Worker & Partition UI

این نفر روی UI عملیاتی روزمره کار می کند.

صفحات اصلی

```
jobs/  
  executions/  
    executions/:id/  
      executions/:id/steps/  
        executions/:id/partitions/  
          workers/  
            worker-groups/  
              sla/
```

مسئولیت‌ها

- نمايش Job Dashboard
- نمايش Execution List
- نمايش Execution Detail
- نمايش Step ها
- نمايش Partition ها
- نمايش Worker Status
- نمايش Worker Group Health
- start / stop / restart از UI
- retry partition از UI

بخش Worker Group در UI

باید این اطلاعات دیده شود:

```
Worker Group Name  
  Domain  
  Status  
  Active Workers  
  Down Workers  
  Busy Workers  
  Max Capacity  
  Current Running Jobs  
  Current Running Partitions  
  Version Distribution  
  Last Heartbeat
```

```
deposit-db-heavy-critical
Status: ACTIVE
Workers: 8 active / 1 down
Running Jobs: 3
Running Partitions: 28 / 32
Version: 1.4.2
```

Definition of Done

- تیم عملیات بتواند وضعیت execution را ببیند.
- وضعیت worker ها و worker group ها مشخص باشد.
- partition progress قابل مشاهده باشد.
- عملیات start/stop/restart/retry از UI ممکن باشد.

Frontend 2: Admin UI, Scheduler UI, Error Center, Reprocess & Audit

این نفر روی UI مدیریتی و پشتیبانی پیشرفته کار می کند.

صفحات اصلی

```
job-definitions/
worker-groups/manage/
schedules/
business-calendar/
dependencies/
errors/
reprocess/
approvals/
audit/
security/roles/
security/permissions/
```

مسئولیت‌ها

- مدیریت Job Definition
- مدیریت Worker Group
- مدیریت allowed jobs برای worker group
- مدیریت schedule
- مدیریت business calendar
- مدیریت dependency
- Error Center
- Reprocess Console
- Approval Console
- Audit Viewer
- Role/Permission UI

Worker Group Admin UI

باید بتوان از UI این کارها را انجام داد:

- ایجاد worker group
- فعال/غیرفعال کردن worker group
- تعیین owner team
- تعیین domain
- تعیین max concurrent jobs
- تعیین max concurrent partitions
- تعیین allowed job ها
- مشاهده worker های عضو
- disable/drain کردن worker

Definition of Done

- admin بتواند worker group تعریف و مدیریت کند.
- schedule ها از UI قابل مدیریت باشند.
- خطاها قابل جستجو باشند.
- reprocess و approval قابل انجام باشد.
- audit عملیات قابل مشاهده باشد.

تقسیم مراحل توسعه با Backend ۶

Phase 0: طراحی و Contract ها

هدف

قبل از کدنویسی، مدل مشترک کل platform مشخص شود.

Team Lead

- معماری کلان
- Core Platform Contract
- Data Model پایه
- State Machine ها
- API Contract ها
- ADR ها

Backend 1

- طراحی Execution Manager
- طراحی Job Execution State Machine
- طراحی Start/Stop/Restart Contract

Backend 2

- طراحی Worker Framework
- طراحی Spring Batch Base
- طراحی Job Template

Backend 3

- طراحی Worker Group Model
- طراحی Worker Registry
- طراحی Worker Heartbeat
- طراحی Worker Routing
- طراحی Capacity Model

Backend 4

- طراحی Partitioning Model
- طراحی Remote Partitioning
- طراحی Partition State Machine

Backend 5

- طراحی Scheduler
- طراحی Business Calendar
- طراحی Dependency Model
- طراحی SLA Model

Backend 6

- طراحی Error Model
- طراحی Audit Model
- طراحی Reprocess Flow
- طراحی RBAC

Frontend 1

- wireframe داشبورد عملیات
- wireframe execution detail
- wireframe worker/partition monitor

Frontend 2

- wireframe admin
- wireframe worker group management
- wireframe scheduler
- wireframe error/reprocess/audit

Phase 1: Platform MVP

هدف

اجرای یک job ساده از UI/API با worker واقعی.

Backend 1

- Execution Request
- Manual Job Start
- Execution State
- Duplicate Lock

Backend 2

- Worker Agent
- Spring Batch Starter
- Sample Job
- Standard Job Parameters

Backend 3

- Worker Group CRUD
- Worker Registration
- Worker Heartbeat
- Worker Routing اولیه

Backend 4

- ساختار اولیه Partition Plan
- Local Partitioning ساده

Backend 5

- BusinessDate Service ساده
- Calendar پایه

Backend 6

- Audit پایه
- Error Event پایه
- RBAC ساده

Frontend 1

- Dashboard اولیه
- Execution List
- Execution Detail
- Worker Monitor

Frontend 2

- Job Definition ساده
- Worker Group Management ساده
- Error List
- Audit List

خروجی Phase 1

- تعریف worker group
- register شدن worker
- اجرای یک job ساده روی worker group مشخص
- مشاهده وضعیت execution
- مشاهده وضعیت worker

Distributed + کامل Phase 2: Worker Group Execution

هدف

اجرای job روی چند VM و کنترل capacity.

Backend 1

- اتصال execution به worker routing
- مدیریت execution های وابسته به worker group

Backend 2

- worker command listener
- اجرای command دریافتی
- graceful stop

Backend 3

- worker capacity management
- worker group limits
- worker drain/disable
- worker version tracking
- stale worker detection

Backend 4

- remote partitioning
- partition assignment
- claim partition
- retry partition
- aggregation

Backend 5

- بررسی capacity قبل از اجرای scheduled/manual job

Backend 6

- permission برای عملیات worker group
- audit worker group actions

Frontend 1

Worker Group Health Dashboard -
Partition Monitor -
retry partition -

Frontend 2

Worker Group Admin کامل -
drain/disable worker از UI -

خروجی Phase 2

- چند worker روی چند VM
- worker group فعال
- اجرای distributed job
- ظرفیت قابل کنترل
- مشاهده health و capacity

Phase 3: Scheduler, Dependency, SLA

هدف

jobها طبق schedule و business calendar اجرا شوند.

Backend 1

- دریافت trigger از scheduler
- تبدیل trigger به execution request

Backend 2

- پشتیبانی worker از scheduled parameters

Backend 3

- اعلام capacity worker group به scheduler

Backend 4

- partition planning برای jobهای scheduled

Backend 5

- scheduler کامل
- cron
- business calendar
- dependency
- EOD/BOD chain
- SLA monitor

Backend 6

- approval برای اجرای دستی job حساس
- audit تغییر schedule

Frontend 1

- SLA dashboard
- dashboard در scheduled/running status

Frontend 2

- Schedule UI
- Business Calendar UI
- Dependency UI
- Approval UI

Phase 4: Error Center, Restart, Reprocess

هدف

پلتفرم برای پشتیبانی واقعی قابل استفاده شود.

Backend 1

- restart job
- stop graceful
- execution recovery

Backend 2

- checkpoint support
- استانداردسازی خطا در worker

Backend 3

- مدیریت restart/retry در worker failure
- تشخیص worker stale و آزادسازی assignment

Backend 4

- retry failed partition
- stale partition recovery

Backend 5

- retry schedule policy
- reschedule در صورت failure مجاز

Backend 6

- error classification
- error center
- reprocess request
- approval reprocess
- audit کامل

Frontend 1

- action های stop/restart/retry
- نمایش failure در execution detail

Frontend 2

- Error Center
- Reprocess Console
- Approval Console
- Audit Viewer

Phase 5: Hardening, Performance, Security

هدف

آماده سازی production.

Backend 1

- تست race condition
- duplicate execution scenarios
- state consistency

Backend 2

- tuning Spring Batch
- tuning connection pool
- worker shutdown/recovery

Backend 3

- worker capacity tuning
- worker group failover scenarios
- rolling deployment support

Backend 4

- partition performance tuning
- 100M+ records تست
- skew detection

Backend 5

- SLA alerting
- scheduler edge cases
- misfire testing

Backend 6

- RBAC کا مل
- masking
- audit hardening
- security test

Frontend 1

- UX dashboard عملیاتی
- auto-refresh
- advanced filters

Frontend 2

- permission-based UI
- audit reports
- reprocess UX

Phase 6: Migration & Rollout

هدف

انتقال تدریجی jobهای واقعی.

Team Lead

- انتخاب jobهای pilot
- هماهنگی با تیمهای دامنه
- مدیریت ریسک rollout

Backend 1

- پشتیبانی job executionهای migrated

Backend 2

- کمک به تیمها برای نوشتن job با template

Backend 3

- تعریف worker groupهای domainهای واقعی
- capacity planning برای هر تیم

Backend 4

- partition strategy برای jobهای حجیم واقعی

Backend 5

- migrated schedule/dependency برای job های migrated

Backend 6

- migrated audit/error/reprocess برای job های migrated

Frontend ها

- گرفتن feedback از عملیات
- بهبود dashboard ها
- اضافه کردن فیلترها و گزارشهای مورد نیاز

ترتیب پیشنهادی Sprint ها

Sprint 0

- Core Platform Contract
- Data Model اولیه
- State Machine ها
- API Contract
- Worker Group Design
- UI Wireframe

Sprint 1

- Control Plane Skeleton
- Worker Skeleton
- Worker Group CRUD
- Worker Registration
- Job Execution ساده
- UI Dashboard اولیه

Sprint 2

- Worker Heartbeat -
- Worker Routing -
- BusinessDate -
- پلا Audit -
- Execution Detail UI -
- Worker Monitor UI -

Sprint 3

- Local Partitioning -
- Partition Plan -
- Worker Capacity -
- Worker Group Limits -
- پلا Error Event -
- Partition UI -

Sprint 4

- Remote Partitioning -
- Worker Command Listener -
- Partition Claim -
- Retry Partition -
- Worker Drain/Disable -

Sprint 5

- پلا Scheduler -
- Business Calendar -
- پلا Dependency -
- Schedule UI -
- Worker Group Capacity Check -

Sprint 6

- Restart
- Graceful Stop
- Error Center
- Reprocess Request
- Approval پايه

Sprint 7

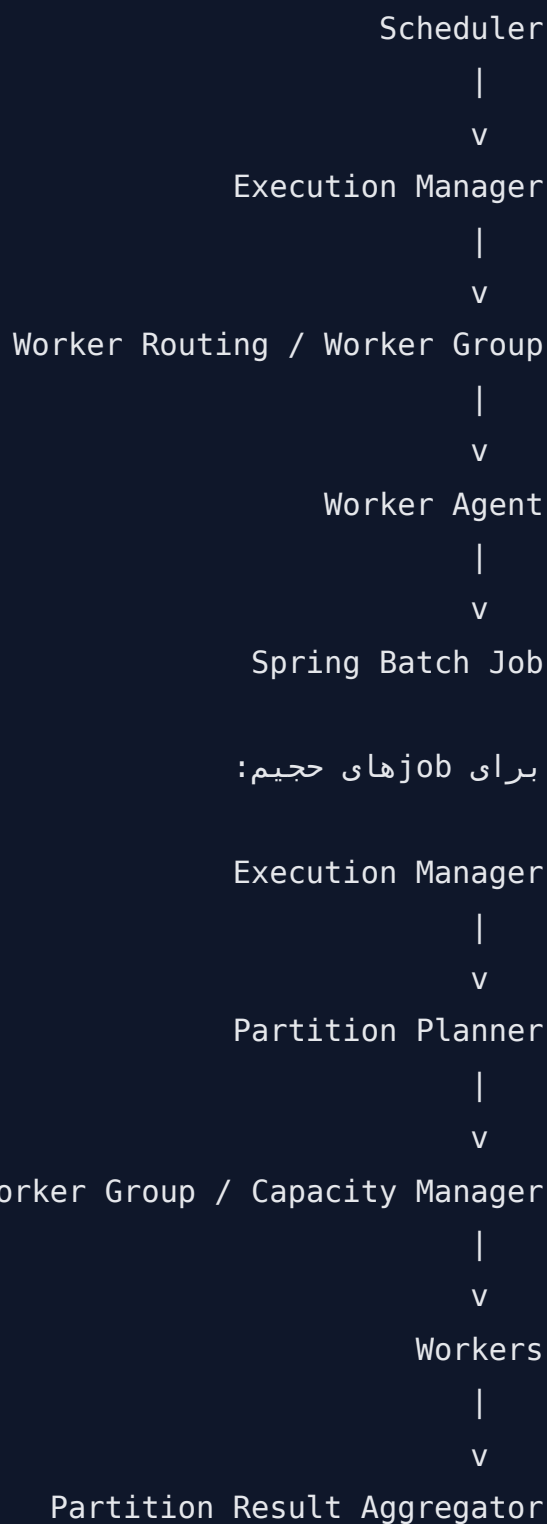
- RBAC کا مل
- Audit Viewer
- SLA Monitoring
- Permission-based UI

+Sprint 8

- Performance Test
- 100M+ benchmark
- Hardening
- Pilot Migration

ارتباط بين ماژولها

نمای کلی ارتباطها:



برای job های حجیم:

Core Platform Contract که باید اول نهایی شود

این مورد برای موازی کاری حیاتی است.

Job Definition Schema	.1
Job Execution State Machine	.2
Partition State Machine	.3
Worker Group Schema	.4
Worker Registration Contract	.5
Worker Heartbeat Contract	.6
Worker Routing Rules	.7
Execution API Contract	.8
Error Event Schema	.9
Audit Event Schema	.10
Business Date Rules	.11
Permission Matrix	.12
Logging Standard	.13
Metrics Standard	.14

پیشنهاد مالکیت نهایی

Team Lead

Architecture, Integration Governance, ADR, Contract

Backend 1

Control Plane, Execution Manager

Backend 2

Worker Framework, Spring Batch Base

Backend 3

Worker Group, Worker Registry, Runtime Management

Backend 4

Sharding, Partitioning, Distributed Execution

Backend 5

Scheduler, Business Calendar, Dependency, SLA

Backend 6

Error, Audit, Reprocess, Security

Frontend 1

Operations Dashboard, Execution, Worker, Partition

Frontend 2

Admin, Scheduler, Worker Group Management, Error, Reprocess, Audit

نکته خیلی مهم درباره Worker Group

در پروژه شما، Worker Group باید از همان MVP وارد شود، نه اینکه بعدا اضافه شود.

اگر از ابتدا Worker Group را لحاظ نکنید، بعدا این مشکلات پیش می‌آید:

- jobها به workerهای اشتباه dispatch میشوند.
- تیمها استقلال واقعی ندارند.
- capacity قابل کنترل نیست.
- deployment تیمها سخت میشود.
- monitoring بر اساس domain ممکن نیست.
- restart و retry در محیط distributed پیچیده میشود.
- عملیات نموداند کدام worker متعلق به کدام تیم است.

پس در MVP حداقل اینها را داشته باشید:

- تعریف worker group
- register شدن worker داخل worker group
- heartbeat worker
- routing job به worker group
- مشاهده وضعیت worker group در UI

بعد در فازهای بعدی کاملترش کنید:

- capacity management
- drain/disable
- version tracking
- allowed jobs
- worker group permissions
- SLA per worker group

جمع بندی

با ۶ Backend، تقسیم بهتر این است:

Backend 1: Execution
Backend 2: Worker Framework
Backend 3: Worker Group / Registry / Runtime
Backend 4: Partitioning / Sharding
Backend 5: Scheduler / Calendar / Dependency / SLA
Backend 6: Error / Audit / Reprocess / Security

و دو Frontend:

Frontend 1: داشبورد عملیاتی، execution, worker, partition
Frontend 2: پنل مدیریتی، scheduler, worker group, error, reprocess, audit

در این مدل، افراد می‌توانند موازی پیش بروند، اما فقط به شرطی که در Sprint 0 این contract ها قفل شوند:

Execution State Machine
Partition State Machine
Worker Group Model
Worker Registration Contract
Job Definition Schema
Error/Audit Schema
API Contract
Permission Matrix

این تقسیم‌بندی هم سرعت توسعه را بالا می‌برد، هم باعث می‌شود کنترل معماری و هماهنگی از بین نرود.